

Sharely

Application Mobile de Reseau Social

Rapport Technique

Stack technologique

Frontend	React Native (Expo) + Expo Router
Backend	NestJS (TypeScript) + TypeORM
Base de donnees	PostgreSQL + Redis
Temps reel	Socket.IO (WebSockets)
Medias	Cloudinary + AWS S3
Auth	JWT + OAuth2 + OTP
IA	OpenAI GPT-4o / Anthropic Claude

Auteurs	Farahe El Montaser & Mehdi Ait El Mehdi
Version	2.0.0
Date	June 2, 2026
Cible	iOS 14+ et Android 8.0+
Statut	En developpement actif

Ce document presente l'architecture, la conception UML, les composants techniques et les corrections apportees en v2.0 du projet Sharely.

Contents

1	Presentation du projet	2
1.1	Organisation du projet	2
2	Objectifs et fonctionnalites	2
3	Conception UML	3
3.1	Diagramme de cas d'utilisation	3
3.2	Diagramme de sequence — Authentification et chargement du feed	4
4	Architecture Backend	5
4.1	Point d'entree — <code>main.ts</code>	5
4.2	Module principal — <code>app.module.ts</code>	6
4.3	Modules fonctionnels	7
5	Architecture Frontend	7
5.1	Composant principal — <code>InstagramHomeScreen</code>	7
5.1.1	Types principaux	7
5.1.2	Hook d'animation du like	7
6	Bugs corriges en v2.0	8
6.1	Bug #1 — Caracteres Unicode affiches en ???	8
6.2	Bug #2 — Boucle infinie dans <code>loadFeed</code>	8
6.3	Bug #3 — Doublons d'annonces au chargement de page	9
6.4	Recapitulatif des corrections	9
7	Stack technique et contraintes	9
8	Installation et execution	9
8.1	Variables d'environnement backend	9
8.2	Lancer le backend	10
8.3	Lancer le frontend	10
9	Phases de developpement	11
10	Conclusion	11

1. Presentation du projet

Sharely est une application mobile de reseau social full-stack, destinee a un public international. Elle permet aux utilisateurs de partager leur quotidien, d'interagir avec leur communaute, de communiquer en temps reel et de decouvrir du contenu personnalise.

Nouveautes v2.0

- Pagination par curseur sur le fil d'actualite
- Stories interactives avec visualisation plein ecran
- Injection d'annonces toutes les 8 publications
- Correction des boucles infinies dans la gestion d'etat React
- Remplacement des caracteres Unicode par des icones vectorielles

1.1. Organisation du projet

```
sharely/
+-- backend/
|   +-- src/
|       +-- main.ts           # Bootstrap NestJS
|       +-- app.module.ts     # Module racine
|       +-- auth/             # JWT + OAuth2 + 2FA
|       +-- users/            # Profils & follows
|       +-- posts/            # Publications & interactions
|       +-- stories/          # Stories ephemerres (24h)
|       +-- chat/             # Messagerie WebSocket
|       +-- chatbot/          # Agent IA
|       +-- notifications/     # Push & in-app
|       +-- ads/              # Publicites
|       +-- media/            # Upload Cloudinary
|       +-- mail/             # Nodemailer
+-- frontend/
    +-- app/                  # Ecrans Expo Router
    +-- components/          # Composants reutilisables
    +-- context/              # ThemeContext, AuthContext
    +-- utils/                # API helpers, socket
```

2. Objectifs et fonctionnalites

Le projet couvre **11 modules fonctionnels** organises en 4 phases de developpement.

Module	Phase	Fonctionnalites principales
M1 — Auth	MVP	Inscription, JWT, 2FA, mot de passe oublie
M2 — Profil	MVP	Photo, bio, statistiques, compte prive
M3 — Publications	MVP	Upload, carrousel, hashtags, sauvegarde
M4 — Feed & Algo	MVP	Fil personnalise, explore, suggestions
M5 — Interactions	MVP	Like, commentaire, follow, partage
M6 — Stories	Phase 2	Stories 24h, sondages, question box
M7 — Reels	Phase 2	Videos courtes 15s-90s, feed dedie
M8 — Messagerie	Phase 3	DM temps reel, groupes, appels
M9 — Notifications	MVP	Push, in-app, WebSockets
M10 — Securite	MVP	Blocage, compte prive, 2FA
M11 — Agent IA	MVP	Assistant, moderation auto, dashboard admin

Table 1: Modules fonctionnels et phases de developpement

3. Conception UML

Cette section presente les deux diagrammes de conception principaux : le diagramme de cas d'utilisation, qui decrit les interactions acteur-systeme, et le diagramme de sequence, qui illustre le flux d'authentification et de chargement du fil d'actualite.

3.1. Diagramme de cas d'utilisation

Le diagramme ci-dessous couvre les cas d'utilisation principaux du MVP. Trois acteurs interagissent avec le systeme : l'**Utilisateur**, l'**Administrateur** et l'**Agent IA**.

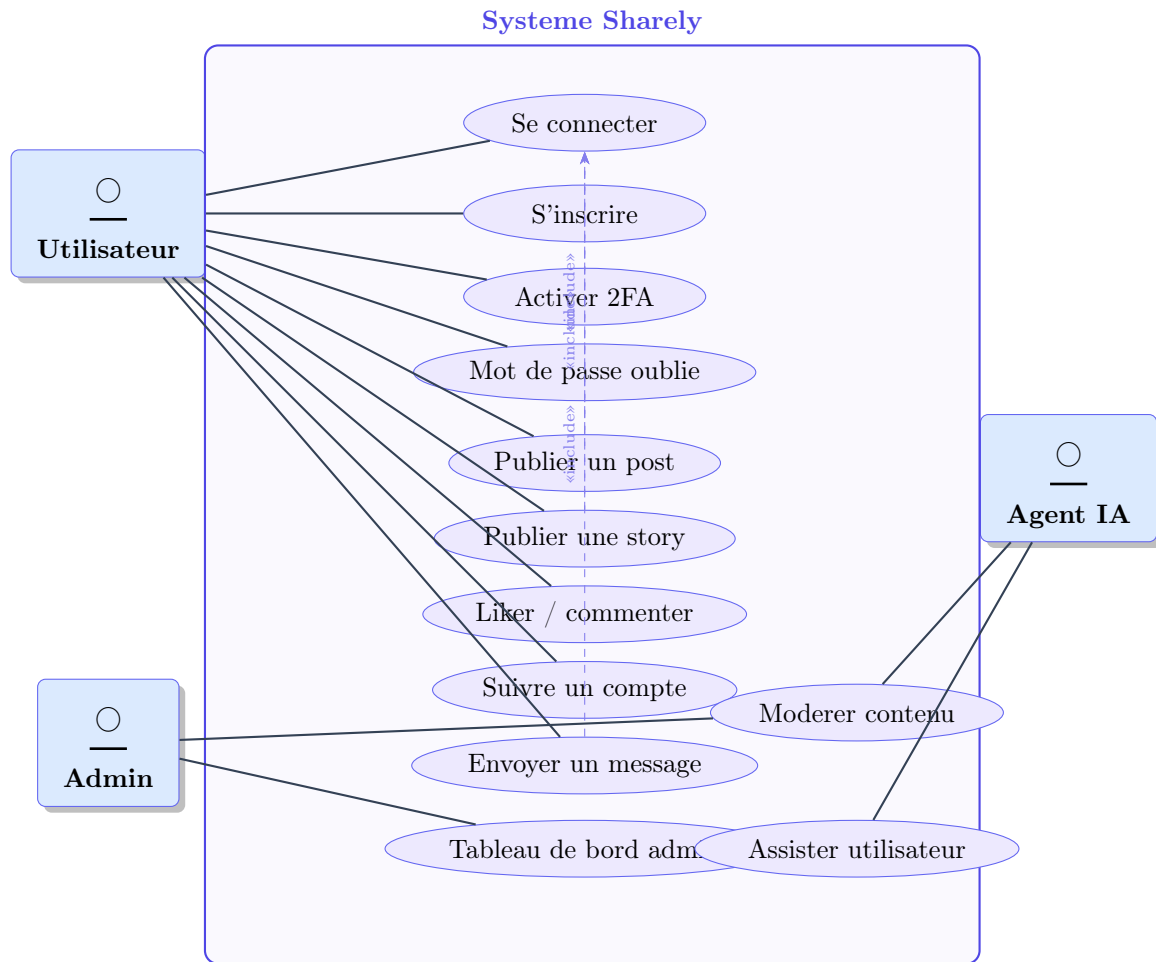


Figure 1: Diagramme de cas d'utilisation — MVP Sharely

3.2. Diagramme de sequence — Authentification et chargement du feed

Ce diagramme illustre deux flux critiques de l'application :

1. Le processus de connexion (JWT + refresh token)
2. Le chargement page du fil d'actualite apres authentication

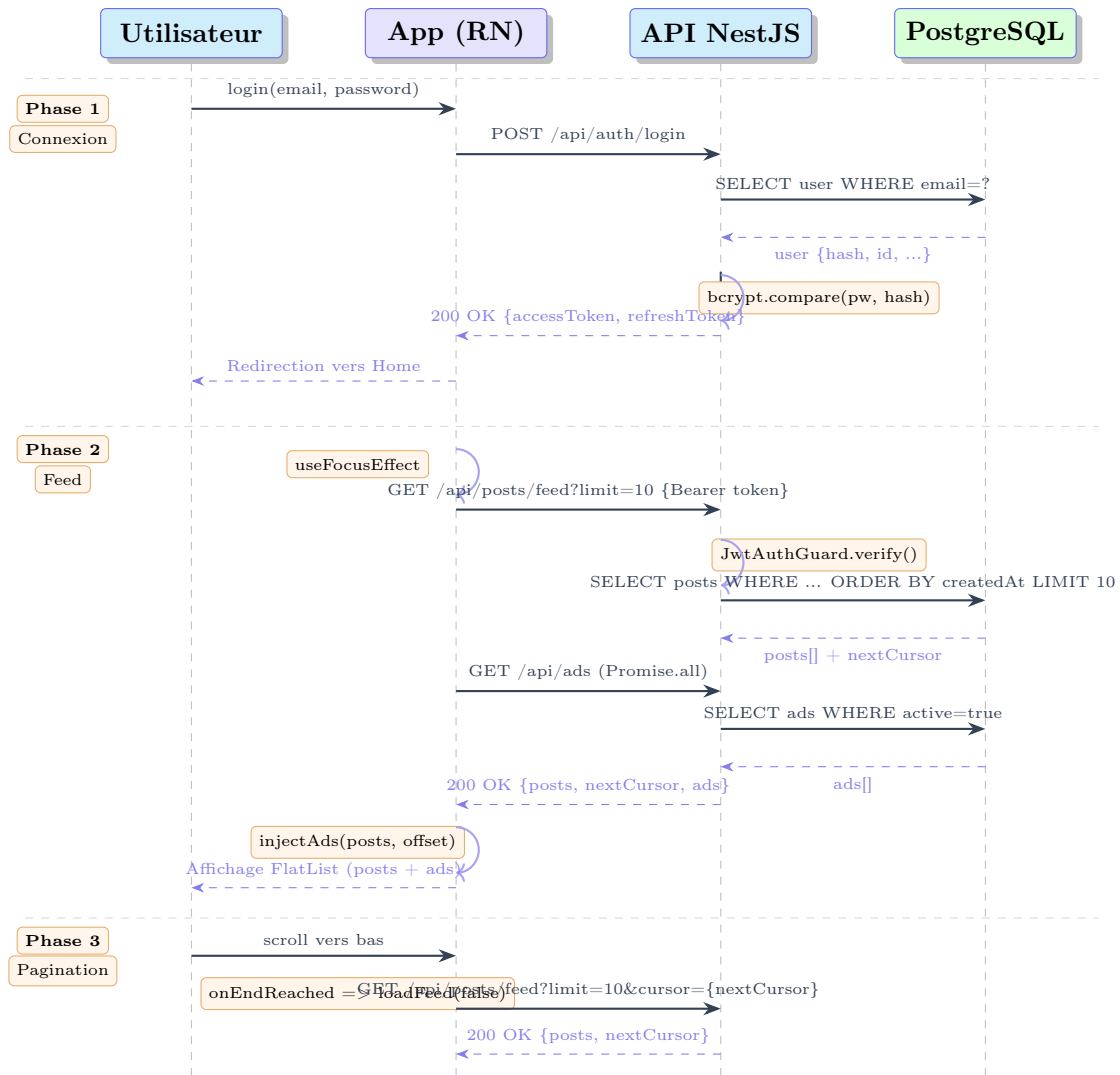


Figure 2: Diagramme de sequence — Connexion, chargement du feed et pagination

4. Architecture Backend

4.1. Point d'entree — main.ts

```

1 import { NestFactory, Reflector } from '@nestjs/core';
2 import { ValidationPipe, ClassSerializerInterceptor } from '@nestjs/
  common';
3 import { AppModule } from './app.module';
4
5 async function bootstrap() {
6   const app = await NestFactory.create(AppModule);
7   app.setGlobalPrefix('api');
8
9   // Validation automatique des DTOs
10  app.useGlobalPipes(new ValidationPipe({
11    whitelist: true,
12    forbidNonWhitelisted: true,
13    transform: true,
14  }));
15
16  // Serialisation (masque les @Exclude dans les entites)

```

```
17 app.useGlobalInterceptors(  
18   new ClassSerializerInterceptor(app.get(Reflector))  
19 );  
20  
21 app.enableCors({  
22   origin: '*',  
23   methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],  
24   allowedHeaders: ['Content-Type', 'Authorization'],  
25 });  
26  
27 await app.listen(process.env.PORT ?? 3000);  
28 }  
29 bootstrap();
```

Listing 1: backend/src/main.ts

4.2. Module principal — app.module.ts

```
1 TypeOrmModule.forRootAsync({  
2   useFactory: (config: ConfigService) => ({  
3     type: 'postgres',  
4     host: config.get('DB_HOST'),  
5     port: config.get<number>('DB_PORT'),  
6     username: config.get('DB_USER'),  
7     password: config.get('DB_PASS'),  
8     database: config.get('DB_NAME'),  
9     entities: [__dirname + '/*.entity{.ts,.js}'],  
10    synchronize: true, // a desactiver en production  
11  }),  
12  inject: [ConfigService],  
13 }),
```

Listing 2: backend/src/app.module.ts (extrait)

Attention — synchronize: true

Desactiver en production et utiliser les migrations TypeORM pour eviter toute perte de donnees lors d'une evolution du schema.

4.3. Modules fonctionnels

Module	Route	Responsabilite
AuthModule	/api/auth	JWT, login, register, 2FA, OAuth2
UsersModule	/api/users	Profil, follow, recherche
PostsModule	/api/posts	CRUD posts, likes, saves, feed pagine
StoriesModule	/api/stories	Creation, consultation, expiration 24h
ChatModule	/api/chat + WS	Messagerie directe WebSocket
ChatbotModule	/api/chatbot	Agent IA (GPT-4o / Claude)
NotificationsModule	/api/notifications	Push FCM, in-app, WebSocket
AdsModule	/api/ads	Liste des publicites actives
MediaModule	/api/media	Upload Cloudinary / AWS S3
MailModule	interne	Nodemailer (reset password, 2FA)

Table 2: Modules NestJS et leurs responsabilites

5. Architecture Frontend

5.1. Composant principal — InstagramHomeScreen

Le composant central orchestre le fil d’actualite, les stories, les publicites et la feuille modale des commentaires.

5.1.1. Types principaux

```

1  type Post = {
2    id: string; username: string; avatar: string;
3    location: string; image: string;
4    likes: number; caption: string; comments: number;
5    timeAgo: string; isLiked: boolean; isSaved: boolean;
6  };
7
8  // Un element du feed = post ou publicite
9  type FeedItem =
10   | { type: 'post'; data: Post; key: string }
11   | { type: 'ad'; data: AdItem; key: string };

```

Listing 3: Types du fil d’actualite

5.1.2. Hook d’animation du like

```

1  function useLikeAnimation() {
2    const scale = useRef(new Animated.Value(1)).current;
3
4    const pulse = () => {
5      scale.setValue(1);
6      Animated.sequence([
7        Animated.spring(scale, {
8          toValue: 1.3, useNativeDriver: true,

```



```

9      speed: 40, bounciness: 12,
10    }),
11    Animated.spring(scale, {
12      toValue: 1, useNativeDriver: true,
13      speed: 20, bounciness: 4,
14    }),
15  ]).start();
16 };
17
18 return { scale, pulse };
19 }

```

Listing 4: useLikeAnimation

6. Bugs corrigés en v2.0

6.1. Bug #1 — Caracteres Unicode affichés en ???

Bug #1 — Icone “more” affichée en ? ? ?

Le caractère Unicode U+2022 (bullet) n’était pas rendu sur certaines polices système Android/iOS.

Avant :

```
1 <Text style={[styles.moreIcon, { color: colors.text }]}>...</Text>
```

Après :

```
1 <Icons name="ellipsis-horizontal" size={20} color={colors.text} />
```

6.2. Bug #2 — Boucle infinie dans loadFeed

Bug #2 — Rechargement infini du fil

cursor et feedItems dans les deps du useCallback provoquaient une boucle : mise à jour de l’état => recreation de la fonction => re-declenchement de useFocusEffect.

Correction — utilisation de refs :

```

1 const cursorRef      = useRef<string | null | undefined>(undefined);
2 const loadingMoreRef = useRef(false);
3 const postCountRef   = useRef(0);
4
5 const loadFeed = useCallback(async (reset = false) => {
6   // lecture via refs => jamais de re-creation du callback
7   if (!reset && cursorRef.current === null) return;
8   if (!reset && loadingMoreRef.current)      return;
9   // ...
10  cursorRef.current      = feedRes.nextCursor;
11  postCountRef.current += feedRes.posts.length;
12 }, []); // deps vides = stable

```

6.3. Bug #3 — Doublons d’annonces au chargement de page

Bug #3 — Annonces dupliquees

La fonction `injectAds` capturerait `feedItems` par closure stale, calculant un offset errone lors de la pagination.

Correction : `postCountRef` remplace le calcul depuis `feedItems`.

```
1 const offset = postCountRef.current;           // valeur courante
2 const newItem = injectAds(feedRes.posts, offset);
3 postCountRef.current += feedRes.posts.length; // mise a jour apres
```

6.4. Recapitulatif des corrections

#	Composant	Description	Solution
1	PostCard	... → ? ? ? sur mobile	Ionicons ellipsis-horizontal
2	HomeScreen	Boucle infinie <code>loadFeed</code>	Refs (<code>cursorRef</code> , <code>loadingMoreRef</code>)
3	HomeScreen	Annonces dupliquees	<code>postCountRef</code> pour l’offset
4	Header	Emojis non rendus	Icones Ionicons

Table 3: Recapitulatif des bugs corriges en v2.0

7. Stack technique et contraintes

Composant	Technologie	Phase	Notes
Mobile (Frontend)	React Native + Expo	V1 & V2	iOS 14+ / Android 8.0+
Backend	NestJS (TypeScript)	V1 & V2	Architecture modulaire
Temps reel	Socket.IO	V1 & V2	Chat + notifications
Appels	Twilio → WebRTC	V1 → V2	Migration prevue en V2
Base de donnees	PostgreSQL + Redis	V1 & V2	Cache Redis pour le feed
Stockage medias	Cloudinary + AWS S3	V1 & V2	Compression + CDN
Push notifications	Firebase FCM	V1 & V2	iOS + Android
Auth	JWT + OAuth2 + OTP	V1 & V2	Google, Apple, SMS
Recherche	Algolia → Elasticsearch	V1 → V2	Migration V2
Agent IA	OpenAI GPT-4o / Claude	V1 & V2	Assistant + moderation
CDN	AWS CloudFront	V1 & V2	Assets statiques

Table 4: Stack technique complete

8. Installation et execution

8.1. Variables d’environnement backend

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASS=motdepasse
DB_NAME=sharely

JWT_SECRET=votre_secret_jwt_fort
JWT_EXPIRES_IN=7d

CLOUDINARY_CLOUD_NAME=...
CLOUDINARY_API_KEY=...
CLOUDINARY_API_SECRET=...

MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USER=votre@email.com
MAIL_PASS=...

PORT=3000
```

Listing 5: backend/.env

8.2. Lancer le backend

```
cd backend && npm install && npm run start:dev
# API disponible sur http://localhost:3000/api
```

8.3. Lancer le frontend

```
cd frontend && npm install && npm start
# Scanner le QR code avec Expo Go
```

Prerequis

- Node.js >= 18 et npm >= 9
- PostgreSQL >= 14 (local ou Docker)
- Expo CLI : `npm install -g expo-cli`
- Application Expo Go sur le telephone (iOS/Android)

9. Phases de developpement

Phase	Objectif	Modules inclus
Phase 1 — MVP	Fondations sociales	Auth, Profil, Posts, Feed & Algo, Interactions, Notifications, Securite de base, Agent IA
Phase 2 — Media	Stories & Reels	Stories (24h, sondages, Q&A), Reels (15-90s), 2FA
Phase 3 — Communication	Messagerie complete	DM temps reel, Appels Twilio, Groupes, Messages vocaux
Phase 4 — Scalabilite	Optimisation & V2	Migration WebRTC, Elasticsearch, MongoDB, Archives, Tendances

Table 5: Phases de developpement du projet

10. Conclusion

La version 2.0 de **Sharely** consolide les fondations techniques du projet. Les principaux apports sont la stabilisation de la gestion d'état asynchrone via les `useRef`, l'élimination des artefacts d'affichage Unicode, et la robustesse du systeme de pagination par curseur.

L'architecture NestJS modulaire et la separation claire frontend/backend permettront une montee en charge progressive selon les phases prevues dans le cahier des charges.

Prochaines etapes prioritaires :

- Passer `synchronize: false` et implementer les migrations TypeORM
- Ajouter les tests unitaires et E2E (Jest, Detox)
- Mettre en place CI/CD (GitHub Actions)
- Implementer l'algorithme de recommandation (Phase 1 — non encore livre)